

# **Reranking (Cross-Encoder / Cohere Rerank)**

---

# **1. Reranking : Cross-Encoder**



# Reranking (Cross-Encoder)

## 1. Reranking이란?

Reranking은 Vector DB에서 검색된 후보 문서를 다시 평가하여 가장 관련성이 높은 순서로 재정렬(Reranking)하는 기술이다.

즉,

1차 검색(Retrieval)

↓

후보 문서 검색

↓

Cross-Encoder로 재평가

↓

최종 순위 결정

이라는 과정을 수행한다.

RAG에서는 검색(Retrieval)과 생성(Generation) 사이에서 검색 품질을 향상시키는 중요한 역할을 한다.

## 2. 왜 Reranking이 필요한가?

일반적인 Vector Search는 Bi-Encoder 기반의 임베딩 유사도만 계산한다.

따라서 의미는 비슷하지만 실제 질문과는 관련성이 낮은 문서가 상위에 검색될 수 있다.

예를 들어,

사용자 질문

RAG에서 *Chunking* 전략이란?

Vector Search 결과

| 순위 | 문서                 |
|----|--------------------|
| 1  | Vector Database    |
| 2  | Embedding          |
| 3  | Chunking           |
| 4  | Prompt Engineering |
| 5  | Agent              |

실제로는

"Chunking"

문서가 가장 관련성이 높지만

Vector Search에서는 3위에 위치할 수도 있다.

Cross-Encoder는 질문과 문서를 함께 비교하여

Chunking

↓

1위

로 다시 정렬한다.

### 3. RAG에서의 위치

사용자 질문



Embedding (Bi-Encoder)



Vector DB 검색



Top-20 후보 문서



Cross-Encoder



Top-5 문서



LLM 답변 생성

즉,

Bi-Encoder는 빠르게 후보 문서를 찾고,

Cross-Encoder는 가장 적절한 문서를 선택한다.

## 4. Cross-Encoder란?

Cross-Encoder는 질문(Query)과 문서(Document)를 하나의 입력으로 함께 넣어 두 문장의 관련성을 직접 계산하는 모델이다.

입력

Question

+

Document



예)

Question

RAG란?

+

Document

RAG(Retrieval-Augmented Generation)는...



출력

관련성 점수

0.98

점수가 높을수록 질문과 문서의 관련성이 높다.

## 5. Bi-Encoder와 Cross-Encoder 차이

### Bi-Encoder

질문과 문서를 각각 임베딩한다.

```
Question
|
Embedding
|
Vector

Document
|
Embedding
|
Vector

↓

Cosine Similarity
```

속도는 매우 빠르지만

문맥을 깊게 비교하지 못한다.

### Cross-Encoder

질문과 문서를 함께 입력한다.

```
Question
+
Document

↓

Cross Encoder

↓

관련성 점수
```

질문과 문서를 동시에 이해하므로

훨씬 정확한 관련성 점수를 계산한다.



## 6. Bi-Encoder와 비교

| 구분           | Bi-Encoder       | Cross-Encoder  |
|--------------|------------------|----------------|
| 입력 방식        | 질문,문서를 각각 인코딩    | 질문과 문서를 함께 입력  |
| 속도           | 매우 빠름            | 느림             |
| 정확도          | 높음               | 매우 높음          |
| 사전 임베딩       | 가능               | 불가능            |
| Vector DB 검색 | 가능               | 불가능            |
| 주요 용도        | 1차 검색(Retrieval) | 재순위(Reranking) |

## 7. 동작 예

사용자 질문

HyDE란 무엇인가?

Vector Search 결과

| 문서                 | 순위 |
|--------------------|----|
| Vector Database 설명 | 1  |
| HyDE 설명            | 2  |
| Agent 설명           | 3  |

Cross-Encoder 계산

| 문서                 | 관련성 점수 |
|--------------------|--------|
| Vector Database 설명 | 0.28   |
| HyDE 설명            | 0.97   |
| Agent 설명           | 0.19   |

Cross-Encoder 계산

| 문서                 | 관련성 점수 |
|--------------------|--------|
| Vector Database 설명 | 0.28   |
| HyDE 설명            | 0.97   |
| Agent 설명           | 0.19   |

최종 결과

| 최종 순위 | 문서                 |
|-------|--------------------|
| 1     | HyDE 설명            |
| 2     | Vector Database 설명 |
| 3     | Agent 설명           |

## 8. 간단한 Python 예제

Python

```
from sentence_transformers import CrossEncoder

# Cross-Encoder 모델
model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L-6-v2"
)

query = "RAG란 무엇인가?"

documents = [
    "RAG는 검색 증강 생성 기술이다.",
    "GPU는 병렬 연산 장치이다.",
    "LangChain은 LLM 프레임워크이다."
]

# 질문-문서 쌍 생성
pairs = [[query, doc] for doc in documents]

# 관련성 점수 계산
scores = model.predict(pairs)

for score, doc in zip(scores, documents):
    print(f"{score:.3f} : {doc}")
```

### 실행 결과

0.982 : RAG는 검색 증강 생성 기술이다.

0.412 : LangChain은 LLM 프레임워크이다.

0.113 : GPU는 병렬 연산 장치이다.

점수가 높은 순서대로 문서를 다시 정렬한다.

## 9. RAG에서 사용하는 방법

일반적인 RAG에서는 다음과 같이 사용한다.

`</>` Python

*# 1차 검색*

```
docs = vectorstore.similarity_search(  
    query,  
    k=20  
)
```

*# Cross-Encoder 재정렬*

```
reranked_docs = reranker(docs)
```

*# 상위 문서만 LLM 전달*

```
top_docs = reranked_docs[:5]
```

즉,

Top-20 검색

↓

Cross-Encoder

↓

Top-5 선정

↓

LLM

의 형태가 가장 일반적이다.

## 10. 언제 사용하는가?

다음과 같은 경우 효과가 크다.

### ① 검색 결과가 많은 경우

Top-50

↓

Top-5 선택

### ② 유사한 문서가 많은 경우

예)

RAG 기초

RAG 심화

RAG 응용

↓

가장 적절한 문서 선택

### ③ 기술 문서 검색

- 논문
- PDF
- 기업 문서
- 매뉴얼

### ④ 질문이 복잡한 경우

2025년에 작성된 LangChain Agent 관련 문서

## 11. 장점

- 질문과 문서를 함께 비교하여 높은 정확도를 제공한다.
- 검색 결과의 Precision을 크게 향상시킨다.
- 기존 Vector DB를 변경하지 않고 적용할 수 있다.
- LLM에 전달되는 문서 품질이 향상된다.

## 12. 단점

① 속도가 느리다.

후보 문서를 하나씩 평가해야 한다.

② 사전 임베딩이 불가능하다.

매번 질문과 문서를 함께 입력해야 한다.

③ 후보 문서가 많으면 시간이 오래 걸린다.

보통 Top-20~Top-50 정도만 재평가한다.

### 13. Bi-Encoder와 Cross-Encoder를 함께 사용하는 이유

두 모델은 서로 경쟁 관계가 아니라 상호 보완 관계이다.

사용자 질문

↓

Bi-Encoder

(빠른 검색)

↓

Top-20 문서

↓

Cross-Encoder

(정밀 재평가)

↓

Top-5 문서

↓

LLM

- Bi-Encoder는 빠른 후보 검색을 담당한다.
- Cross-Encoder는 정확한 문서 선별을 담당한다.

이 조합이 현재 대부분의 RAG 시스템에서 사용하는 표준적인 검색 구조이다.

## 14. 장단점 요약

### 장점

- 질문과 문서를 함께 분석하여 높은 정확도를 제공한다.
- 검색 결과의 Precision을 크게 향상시킨다.
- LLM의 답변 품질을 높인다.
- 기존 RAG 파이프라인에 쉽게 추가할 수 있다.

### 단점

- 추론 속도가 느리다.
- 후보 문서를 모두 평가해야 하므로 계산량이 증가한다.
- 대규모 문서 전체 검색에는 사용할 수 없고, Top-K 후보 문서에만 적용한다.

## 15. 한 줄 요약

Cross-Encoder 기반 Reranking은 "Vector DB에서 검색된 후보 문서를 질문과 함께 입력하여 관련성을 다시 계산하고, 가장 적합한 순서로 재정렬하는 기술"로, RAG 시스템의 검색 정확도(Precision)를 크게 향상시키는 대표적인 후처리(Search Refinement) 기법이다.



## **2. Reranking : Cohere Rerank**



## Cohere Rerank 소개

Cohere Rerank는 기존 검색 결과의 순위를 다시 매겨 AI가 찾아내는 정보의 정확도를 극대화해 주는 유료 정렬 전문 API입니다.

### 왜 필요한가요? (핵심 개념)

기존 키워드 검색이나 일반 벡터 검색은 사용자의 질문 의도와 조금 달라도 단어가 비슷하면 상위 결과로 끌어옵니다.

- **기존 검색:** "일단 관련되어 보이는 문서 50개를 찾음" (불필요하거나 덜 중요한 정보 섞임)
- **Cohere Rerank 적용:** "50개 문서를 문맥까지 정밀하게 읽고, 진짜 답이 되는 순서대로 상위 1~3개를 다시 정렬함"

### 요금 및 가격 구조

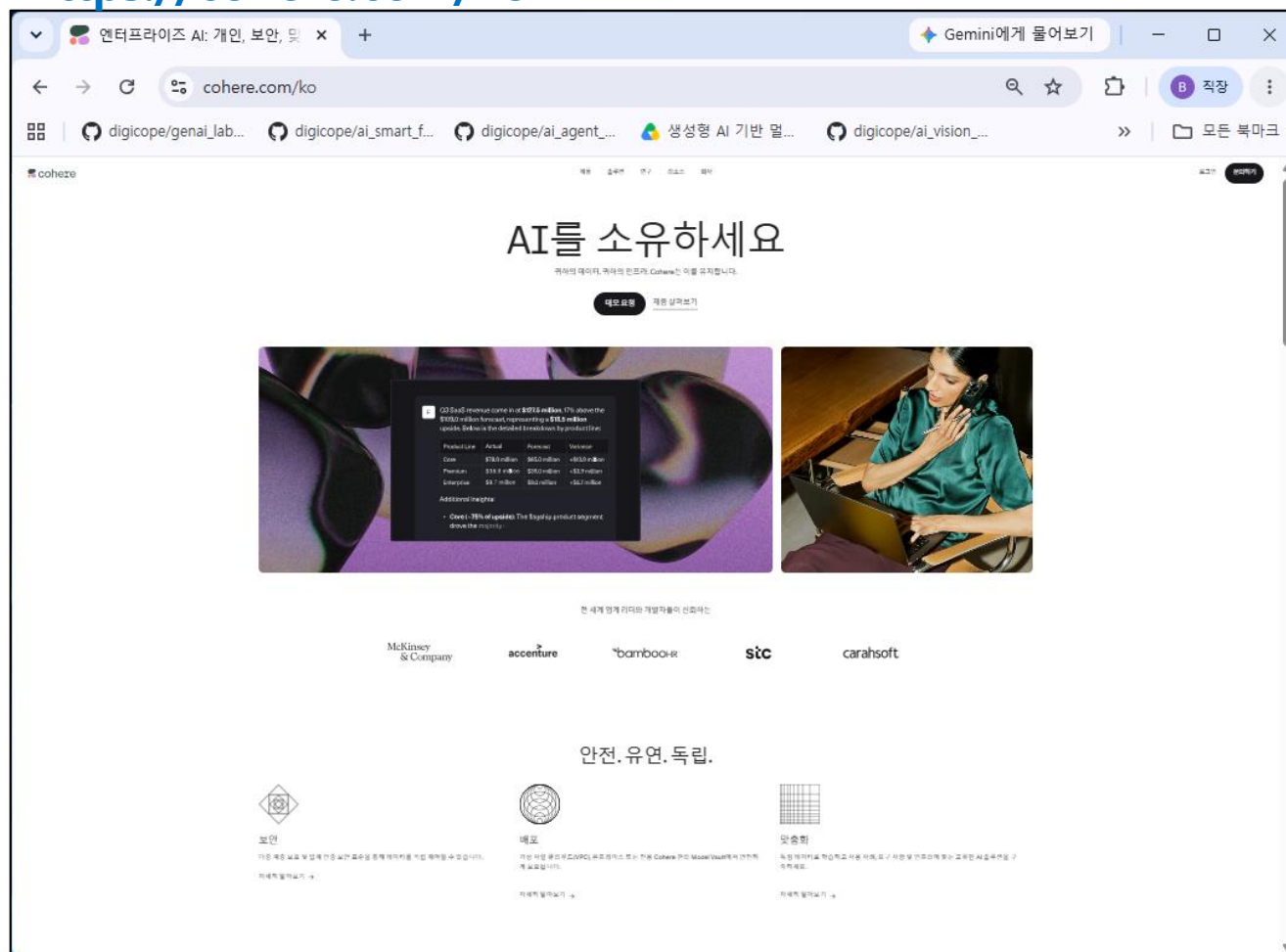
- **무료 체험:** 가입 시 제공되는 Trial Key로 월 1,000회까지 무료 테스트 가능 (속도 제한 및 비상업용)
- **유료 상용 플랜 (Pay-as-you-go):** 실서비스 적용 시 1,000회 검색 요청당 \$2.00 수준의 종량제 요금이 부과됩니다.

# Cohere Rerank API Key 발급 방법

## 1단계. Cohere 회원가입

아래 사이트에 접속하여 Google, GitHub 또는 이메일 계정으로 회원가입한다.

<https://cohere.com/ko>



## Log in

 Continue with Google

 Continue with Github

EMAIL

yourname@email.com

PASSWORD

\*\*\*\*\*



[Forgot Password](#)

Log in

→

By signing up, you agree to the [Terms of Use](#) and [Privacy Policy](#).

[New user? Sign up](#)

설문에 필요한 해당 사항을 기입하고 [Submit]버튼을 누른다

123

## Let's get to know you better

FIRST NAME  
BH

LAST NAME  
GO

☒ I would like to receive email updates about new products or services, newsletters and/or promotional or marketing materials from cohere. I may withdraw my consent at any time by unsubscribing.

Continue→

123

## What is your role?

Please select one option.

|                           |                   |
|---------------------------|-------------------|
| Machine Learning Engineer | Software Engineer |
| Data Scientist            | Researcher        |
| Product Manager           | Student           |

Specify anything else below.

eg. Marketing Specialist - Chief Wizard

Continue→

BackSkip this step

123

## What do you plan to do with Cohere?

Please select all options that apply.

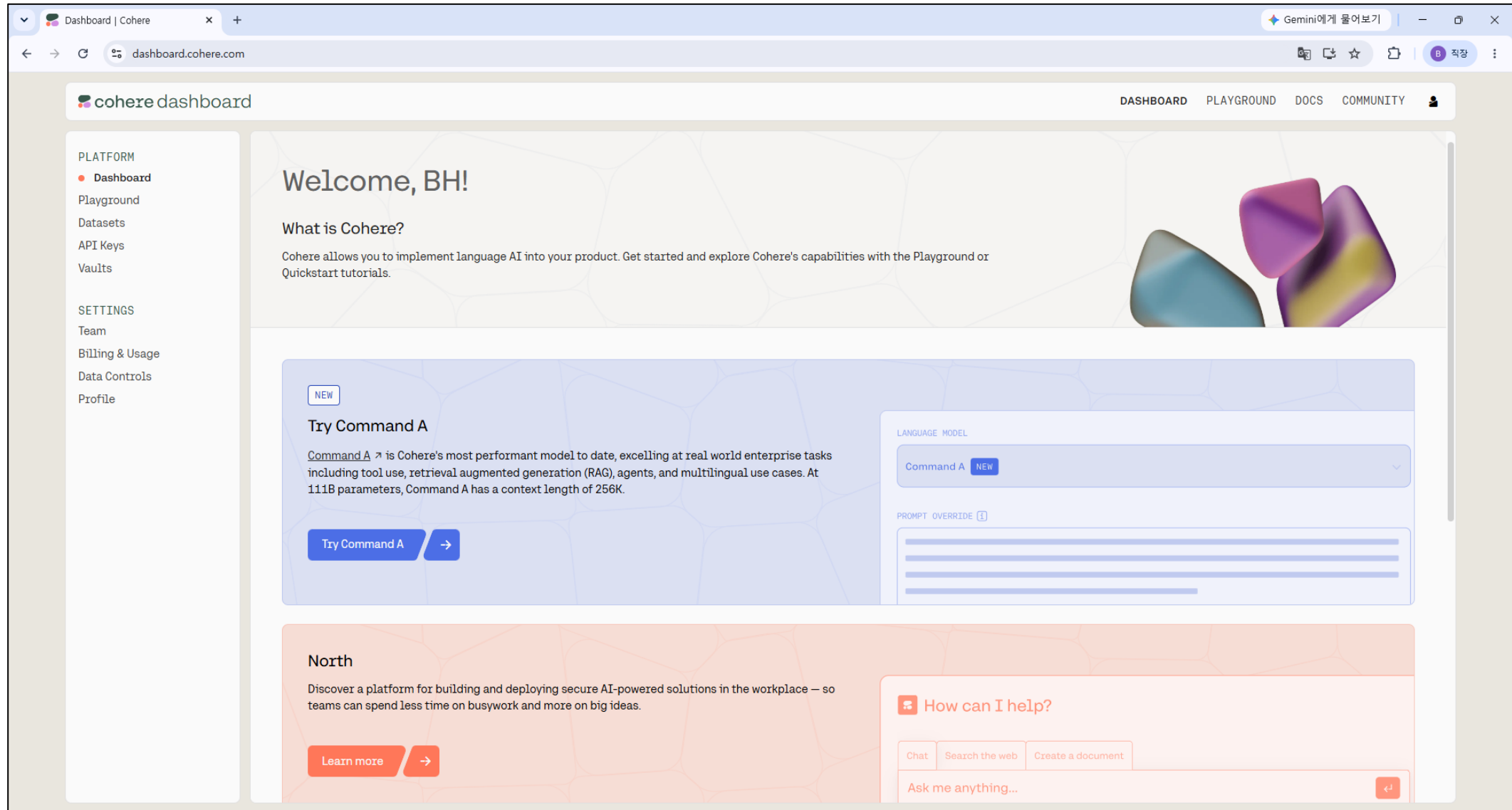
|                    |                         |
|--------------------|-------------------------|
| Content Moderation | Text Classification     |
| Semantic Search    | User Intent Recognition |
| Text Generation    | Entity Extraction       |
| Text Summarization | Chat                    |

Specify anything else below.

e.g. Make Chatbots Sound Human

Submit→

회원 가입 완료 후 화면에서 좌측 메뉴에서 [API Keys]를 클릭한다



Trial API Key가 자동으로 미리 생성되어 있으며, 필요하면 새로운 Trial Key를 생성할 수도 있다.

생성된 API Key를 메모장에 복사하여 저장해둔다.

.env 파일에 아래와 같이 추가  
**COHERE\_API\_KEY="WDQwaN1mMXDeo67mAXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"**

default



WDQwaN1mMXDeo67mAXXXXXXXXXXXXXXXXXXXXX



API Keys | Cohere

dashboard.cohere.com/api-keys

cohere dashboard

DASHBOARD PLAYGROUND DOCS COMMUNITY

PLATFORM

- Dashboard
- Playground
- Datasets
- API Keys
- Vaults

SETTINGS

- Team
- Billing & Usage
- Data Controls
- Profile

## API Keys

You're using a Cohere Free Account

Free usage of Cohere's API keys and applications is free, but limited. In order to use Cohere's API and applications without trial limitations just complete a short application form (which helps us ensure the safety of our community and the end users impacted by Cohere's models).

Get your Production key →

Trial keys **FREE** + New Trial key

Calls made using Trial keys are free of charge. Trial keys are rate-limited, and cannot be used for commercial purposes.

| NAME    | KEY      | CREATED      | CREATOR               |
|---------|----------|--------------|-----------------------|
| default | ••••Tb8D | Jul 31, 2026 | digicope@aicore.co.kr |

# Cohere Rerank 무료(Trial) API Key 사용 제한

현재(2026년 기준) Cohere Trial API Key의 주요 제한은 다음과 같다.

| 항목            | Trial 제한                      |
|---------------|-------------------------------|
| 사용 요금         | 무료                            |
| 월간 호출 수       | 1,000 API Calls/월 (모든 API 합산) |
| Rerank API    | 10 requests/minute (RPM)      |
| Embed API     | 2,000 inputs/minute           |
| Chat API      | 20 requests/minute            |
| Production 사용 | 불가 (개발/테스트 전용)                |

## Cohere Rerank API 요금

현재(2026년 기준) Cohere의 Self-Serve API 기준 Rerank 모델 가격은 다음과 같다.

| 모델            | 가격                   |
|---------------|----------------------|
| Rerank v3     | 입력 토큰 100만 개당 \$2.00 |
| Rerank v3.5   | 입력 토큰 100만 개당 \$2.00 |
| Rerank 4 Fast | 입력 토큰 100만 개당 \$2.00 |
| Rerank 4 Pro  | 입력 토큰 100만 개당 \$2.50 |

참고로 Rerank 모델은 생성형 LLM과 달리 출력 토큰 비용이 없으며, 검색 대상 문서와 질의(Query)에 포함된 입력 토큰만 과금된다.  aipricing.guru +1



# Reranking (Cohere Rerank)

## 1. Cohere Rerank란?

Cohere Rerank는 Cohere에서 제공하는 Reranking 전용 API 서비스이다.

Vector DB에서 검색된 후보 문서를 Cohere의 Rerank 모델이 다시 평가하여 질문과 가장 관련성이 높은 순서로 재정렬한다.

즉,

1차 검색(Retrieval)

↓

후보 문서 검색

↓

Cohere Rerank

↓

관련성 점수 계산

↓

최종 순위 결정

이라는 과정을 수행한다.

## 2. 왜 Cohere Rerank를 사용하는가?

Vector Search는 임베딩 벡터의 유사도를 이용하여 검색하므로 검색 속도는 매우 빠르지만, 실제 의미적 관련성이 가장 높은 문서가 항상 상위에 위치하는 것은 아니다.

예를 들어,

사용자 질문

RAG에서 `Chunking` 전략이란?

Vector Search 결과

| 순위 | 문서                 |
|----|--------------------|
| 1  | Vector Database    |
| 2  | Embedding          |
| 3  | Chunking           |
| 4  | Prompt Engineering |
| 5  | Agent              |

실제로는 "Chunking" 문서가 가장 적합하지만 3위에 위치할 수도 있다.

Cohere Rerank는 질문과 후보 문서를 다시 비교하여

Chunking

↓

1위

로 재정렬한다.

### 3. 동작 과정

사용자 질문

↓

Embedding (Bi-Encoder)

↓

Vector DB 검색

↓

Top-20 후보 문서

↓

Cohere Rerank API

↓

↓

관련성 점수 계산

↓

Top-5 문서

↓

LLM 답변 생성

## 4. Cohere Rerank의 특징

Cohere Rerank는 Cross-Encoder 기반으로 동작하는 클라우드 API이다.

사용자는 모델을 직접 설치하거나 학습할 필요 없이 API만 호출하면 된다.

주요 특징

- Cross-Encoder 기반의 높은 정확도
- API 방식으로 간편한 사용
- 다양한 언어 지원
- LangChain과 손쉬운 연동
- 별도의 GPU 환경이 필요 없음

## 5. Cross-Encoder와 Cohere Rerank의 관계

Cross-Encoder는 Reranking 모델의 구조를 의미한다.

Cohere Rerank는 Cross-Encoder 구조를 API 형태로 제공하는 서비스이다.

즉,

Cross-Encoder



관련성 계산 방식



Cohere Rerank



Cross-Encoder 기반 API 서비스

## 6. 동작 예

사용자 질문

HyDE란 무엇인가?

Vector Search 결과

| 문서                 | 순위 |
|--------------------|----|
| Vector Database 설명 | 1  |
| HyDE 설명            | 2  |
| Agent 설명           | 3  |

Cohere Rerank 계산

| 문서                 | 관련성 점수 |
|--------------------|--------|
| Vector Database 설명 | 0.26   |
| HyDE 설명            | 0.99   |
| Agent 설명           | 0.18   |

Cohere Rerank 계산

| 문서                 | 관련성 점수 |
|--------------------|--------|
| Vector Database 설명 | 0.26   |
| HyDE 설명            | 0.99   |
| Agent 설명           | 0.18   |

최종 결과

| 최종 순위 | 문서                 |
|-------|--------------------|
| 1     | HyDE 설명            |
| 2     | Vector Database 설명 |
| 3     | Agent 설명           |

## 7. 간단한 Python 예제

```
</> Python

from cohere import ClientV2

co = ClientV2(api_key="YOUR_API_KEY")

documents = [
    "RAG는 검색 증강 생성 기술이다.",
    "GPU는 병렬 연산 장치이다.",
    "LangChain은 LLM 프레임워크이다."
]

response = co.rerank(
    model="rerank-v3.5",
    query="RAG란 무엇인가?",
    documents=documents
)

for result in response.results:
    print(
        documents[result.index],
        result.relevance_score
    )
```

### 실행 결과

RAG는 검색 증강 생성 기술이다. 0.99

LangChain은 LLM 프레임워크이다. 0.41

GPU는 병렬 연산 장치이다. 0.12

점수가 높은 순서대로 문서를 재정렬한다.

## 8. LangChain에서 사용하기

LangChain에서는 `CohereRerank` 클래스를 이용하여 쉽게 사용할 수 있다.

Python

```
from langchain_cohere import CohereRerank
from langchain.retrievers import ContextualCompressionRetriever

compressor = CohereRerank(
    model="rerank-v3.5"
)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectorstore.as_retriever()
)

docs = compression_retriever.invoke(
    "RAG란 무엇인가?"
)
```

내부적으로

Retriever

↓

Top-K 문서

↓

Cohere Rerank

↓

상위 문서 선택

과정을 자동으로 수행한다.



## 9. 언제 사용하는가?

다음과 같은 경우 매우 효과적이다.

### ① 검색 결과가 많은 경우

Top-50



Top-5 선정

---

### ② 비슷한 문서가 많은 경우

예)

RAG 기초

RAG 심화

RAG 응용



가장 관련성이 높은 문서 선택

### ③ 기술 문서 검색

- PDF
  - 논문
  - 기업 문서
  - 매뉴얼
- 

### ④ 고객지원 FAQ

수백 개의 FAQ 중 가장 적절한 답변 선택

---

### ⑤ 기업 내부 RAG

사내 문서 검색

규정 검색

매뉴얼 검색

## 10. 장점

- 높은 검색 정확도(Precision)를 제공한다.
- 별도의 모델 학습이 필요 없다.
- API만으로 쉽게 사용할 수 있다.
- LangChain과 쉽게 통합된다.
- GPU가 없어도 사용할 수 있다.

## 11. 단점

① API 비용이 발생한다.

호출량이 많을수록 비용이 증가한다.

② 인터넷 연결이 필요하다.

클라우드 API이므로 오프라인 환경에서는 사용할 수 없다.

③ 응답 시간이 증가한다.

후보 문서를 다시 평가해야 한다.

④ 후보 문서 수가 많을수록 시간이 오래 걸린다.

일반적으로 Top-20~Top-50 정도만 재정렬한다.

## 12. Cross-Encoder와 비교

| 구분      | 직접 Cross-Encoder | Cohere Rerank |
|---------|------------------|---------------|
| 실행 방식   | 로컬 모델 실행         | 클라우드 API      |
| GPU 필요  | 권장               | 불필요           |
| 모델 관리   | 직접 관리            | 불필요           |
| 설치      | 필요               | 불필요           |
| 사용 편의성  | 보통               | 매우 쉬움         |
| 비용      | GPU 비용           | API 비용        |
| 오프라인 사용 | 가능               | 불가능           |

## 13. RAG에서의 위치

사용자 질문

↓

Bi-Encoder

↓

Vector DB 검색

↓

Top-20 후보 문서

↓

Cohere Rerank

↓

Top-5 문서

↓

LLM

Cohere Rerank는 검색(Retrieval)과 생성(Generation) 사이에서 가장 관련성이 높은 문서를 선별하는 역할을 수행한다.

## 14. 장단점 요약

### 장점

- Cross-Encoder 수준의 높은 검색 정확도를 제공한다.
- API 방식으로 간편하게 사용할 수 있다.
- GPU 없이도 고품질 Reranking이 가능하다.
- 기존 RAG 시스템에 쉽게 추가할 수 있다.

### 단점

- API 사용 비용이 발생한다.
- 인터넷 연결이 필요하다.
- 응답 시간이 다소 증가한다.
- 대규모 문서 전체가 아닌 Top-K 후보 문서에만 적용하는 것이 일반적이다.

## 15. 한 줄 요약

Cohere Rerank는 "Vector DB에서 검색된 후보 문서를 Cohere의 Cross-Encoder 기반 Reranking API로 다시 평가하여 가장 관련성이 높은 순서로 재정렬하는 서비스"로, 별도의 모델 설치 없이 RAG 시스템의 검색 정확도(Precision)를 크게 향상시킬 수 있는 대표적인 상용 Reranking 솔루션이다.



감사합니다